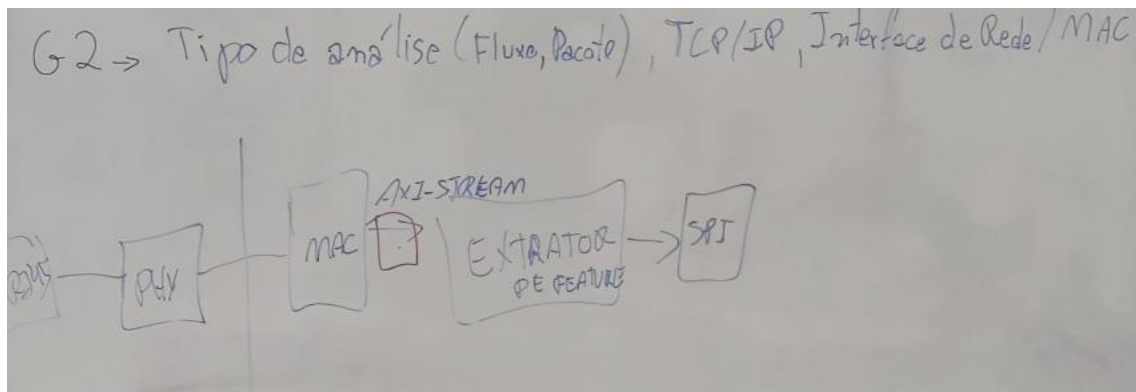


Pesquisa da parte de hardware:



1. Recepção e parsing de pacotes

1.1 — Fakernet: *small and fast FPGA-based TCP and UDP communication*

(<https://arxiv.org/pdf/2003.12527>)

Esse trabalho apresenta uma implementação de comunicação TCP/UDP diretamente em FPGA, com o objetivo de transformar a FPGA de um sistema de aquisição de dados em uma interface Ethernet capaz de transmitir e receber dados. A principal preocupação dos autores foi conseguir uma implementação pequena em recursos e suficientemente rápida para saturar uma conexão de 1 Gb/s. Uma decisão importante foi trabalhar internamente com **palavras de 16 bits**, reduzindo a frequência necessária para processar os dados. ([arXiv](https://arxiv.org/pdf/2003.12527))

Na implementação de hardware, o processamento é feito de forma orientada a fluxo. O pacote entra na FPGA e os dados são processados enquanto estão chegando, em vez de primeiro armazenar todo o pacote e depois fazer o parsing. O projeto utiliza máquinas de estados para controlar as diferentes etapas do processamento e memórias para armazenar dados que precisam permanecer disponíveis entre as etapas.

O trabalho é interessante pela ideia de **processamento streaming**. Ou seja, o parser não precisa esperar o pacote inteiro para começar a interpretar os campos. Sendo, isso pode ser aproveitado para que, conforme os bytes do Ethernet/IPv4/TCP/UDP chegam, os campos necessários para a 5-tupla já sejam capturados.

1.2 — UDP Parser MII v5 — (<https://github.com/adilsondias-engineer/06-fpga-udp-parser-mii-v5>)

O projeto foi implementado em uma **Xilinx Artix-7 XC7A100T**, utilizando a placa Arty A7-100T e o PHY DP83848J. O PHY fornece os dados através de **MI**, com um domínio de clock de 25 MHz. Depois, o processamento interno utiliza um clock de 100 MHz.

O primeiro bloco é o receptor MII. Ele recebe os nibbles de 4 bits provenientes do PHY e os transforma em bytes. Também remove o preâmbulo e identifica o início do frame através do SFD.

Depois entra o **MAC parser**, que:

- identifica o endereço MAC;
- verifica o endereço de destino;
- extrai o EtherType;
- mantém um contador da posição do byte;
- indica quando existe um frame válido.

Depois vem o **IP parser**, que verifica:

- versão IPv4;
- IHL;
- checksum;
- protocolo;
- endereço IP de origem;
- endereço IP de destino;
- tamanho total do pacote.

Finalmente, o UDP parser captura as portas de origem e destino e valida o tamanho do datagrama.

Uma coisa muito importante aconteceu nesse projeto: versões anteriores apresentavam problemas de sincronização entre os módulos. O parser UDP dependia de sinais que chegavam em momentos diferentes, produzindo uma taxa de sucesso de apenas aproximadamente 1%. A versão 5 foi reescrita para fazer o parsing **byte a byte em tempo real**, sem depender do sinal `ip_valid` para iniciar o parser UDP. Isso levou a uma taxa de sucesso de 100% nos testes apresentados.

Outro ponto importante é o **Clock Domain Crossing**. Como o PHY trabalha em 25 MHz e o restante do sistema em 100 MHz, os autores utilizam sincronizadores de dois flip-flops para sinais de um bit e fazem a captura de sinais multibit condicionada por sinais de validade sincronizados.

O projeto utiliza praticamente somente lógica programável: cerca de 2.500 células lógicas, aproximadamente 1.800 LUTs, 1.200 flip-flops e **0 BRAM**, segundo os dados do próprio repositório.

1.3 — Blue Ethernet (<https://medium.com/@datenlord/blue-ethernet-high-performance-fpga-network-packet-processing-project-5d66c7f59b7a>)

O Blue Ethernet é uma biblioteca de componentes de Ethernet para FPGA. Diferentemente do projeto anterior, ele é construído em **Bluespec SystemVerilog**, mas disponibiliza componentes que podem ser utilizados na geração de hardware/Verilog.

O conceito central é trabalhar com **streams**, em que os dados são acompanhados por sinais de controle `valid` e `ready`. Uma transferência só ocorre quando ambos estão ativos.

O projeto trabalha internamente com um DataStream de **256 bits** e possui adaptadores para transformar esse fluxo em um **AXI-Stream de 512 bits**, utilizado pelas interfaces Ethernet da Xilinx.

Isso é muito relevante para a arquitetura de vocês porque a imagem que você recebeu possui justamente:

MAC → AXI-Stream → Feature Extractor.

No Blue Ethernet, o parser recebe o stream Ethernet e separa os **headers dos payloads**. Existem módulos específicos para adicionar ou remover cabeçalhos do stream.

Para UDP/IP/Ethernet, os metadados do pacote são separados dos dados propriamente ditos. Assim, em vez de passar todo o pacote para o próximo módulo, o parser pode entregar informações estruturadas como endereço IP, portas etc.

Isso permite que outro módulo trabalhe somente com os metadados.

1.4 — Low-Latency Modular Packet Header Parser for FPGA

(https://www.fit.vut.cz/research/result-file/c97063/278875/korenek_12_Q4_01.pdf)

Esse trabalho apresenta um parser de pacotes modular e pipeline capaz de ultrapassar **100 Gb/s**, com baixa latência. O diferencial não é simplesmente fazer Ethernet/IP/TCP, mas criar uma arquitetura em que os diferentes protocolos possam ser adicionados de maneira modular.

Os autores implementam o parser diretamente em **VHDL**, buscando uma estrutura uniforme e facilmente extensível.

A ideia é que cada protocolo seja tratado como uma etapa do processamento. O parser analisa o protocolo atual, identifica qual protocolo vem em seguida e encaminha as informações para o próximo estágio.

O ponto mais importante para vocês é o uso de uma arquitetura **pipeline**. Em vez de existir uma única FSM que faça todo o parsing sequencialmente, diferentes partes do parser podem estar trabalhando simultaneamente em diferentes pacotes.

Isso é fundamental em redes de alta velocidade.

Por exemplo, enquanto o módulo está terminando de analisar o TCP de um pacote, outro estágio pode estar analisando o IPv4 do próximo pacote.

O trabalho permite ajustar a relação entre latência e throughput, justamente porque o número de estágios pode ser utilizado para adaptar a arquitetura à aplicação.

Ethernet → IPv4 → TCP/UDP.

1.5 — ITCH Parser — (<https://github.com/adilsondias-engineer/07-fpga-itch-parser>)

Esse projeto não trabalha com TCP/IP como aplicação final, mas é muito útil porque mostra uma segunda camada de parsing depois de UDP.

A implementação utiliza a mesma base do projeto anterior: **MII → MAC → IPv4 → UDP**, e adiciona um `itch_parser` depois do UDP.

O hardware foi implementado em uma **Artix-7 XC7A100T**, com PHY DP83848J e Vivado.

O parser ITCH funciona com uma FSM simples. Primeiro identifica o tipo da mensagem, depois consulta o tamanho esperado e passa a contar os bytes. Conforme o contador chega às posições determinadas pelo protocolo, os campos são capturados.

São extraídas informações diferentes conforme o tipo de mensagem, como:

- Order Reference;
- Buy/Sell;
- Shares;
- Symbol;
- Price;
- Match Number etc.

A FSM possui estados como IDLE, READ_TYPE, COUNT_BYTES, COMPLETE e ERROR.

1.6 — A High-Performance FPGA-Based RoCE v2 RDMA Packet Parser and Generator (<https://www.mdpi.com/2079-9292/13/20/4107>)

Esse trabalho é muito interessante para a parte de **pipeline + AXI-Stream + processamento em hardware**.

O sistema foi implementado em **Verilog RTL** sobre uma **Xilinx Alveo U200** e suporta processamento próximo de 100 Gb/s.

No caminho de recepção, os pacotes entram através de **AXI-Stream**. O projeto utiliza uma série de módulos independentes em pipeline.

Um dos principais módulos é o **Packet Classifier**. Ele verifica o pacote, calcula os checksums e classifica o tipo de pacote.

Para cálculos CRC, os autores utilizam **LFSR — Linear Feedback Shift Register**, implementado em hardware.

Depois existe um **Packet Splitter**, que lê informações do cabeçalho e separa os diferentes tipos de mensagens. Os cabeçalhos separados são armazenados em filas/cache apropriadas.

O ponto importante é que cada módulo trabalha de maneira relativamente independente dentro do pipeline.

1.7 — The Design of a Dynamic Configurable Packet Parser Based on FPGA (<https://www.mdpi.com/2072-666X/14/8/1560>)

Esse trabalho é mais sofisticado que os parsers anteriores.

Em vez de implementar rigidamente:

Ethernet → IPv4 → TCP → UDP

no código RTL, os autores criam um parser que pode ser **configurado dinamicamente**.

A arquitetura utiliza três módulos principais de análise de protocolo juntamente com **TCAM + SRAM**.

A ideia é que as regras de parsing fiquem armazenadas nas memórias. Assim, quando aparece um novo protocolo ou uma nova estrutura de cabeçalho, não é necessário necessariamente alterar o código Verilog.

O TCAM é utilizado para fazer o **matching** das informações do pacote com as regras de parsing, enquanto a SRAM armazena informações associadas àquela regra.

Os autores utilizam uma estrutura de "chain mapping" para representar o grafo de protocolos. Cada entrada possui informações como:

- protocolo;
- existência do próximo protocolo;
- offset;
- tamanho;
- regras de extração;
- posição onde o campo será armazenado.

Os campos extraídos são organizados em buffers de 8, 16 e 32 bits.

Outra característica interessante é que a configuração pode vir do **control plane através de mensagens FlowMod**, permitindo modificar as regras de parsing sem recompilar toda a lógica.

A implementação foi feita em Xilinx UltraScale+ e alcançou mais de 80 Gb/s, com latência máxima de 36 ns para parsing até L4.

Para o TCC: conceitualmente excelente, mas provavelmente mais complexa do que vocês precisam. A arquitetura fixa Ethernet/IPv4/TCP/UDP pode ser mais simples e adequada para um TCC.

1.8 — FPGA-Packet-Parser — (<https://github.com/Saadkalim0173/FPGA-Packet-Parser>)

Esse projeto é diferente dos anteriores porque seu maior valor está na **verificação**, não na arquitetura de rede.

Ele implementa um parser sintetizável em SystemVerilog que recebe um stream de 8 bits utilizando valid/ready. O parser possui:

- header shift register;
- FSM de parsing;
- contador de payload;

- cálculo CRC16;
- extração de metadata;
- controle valid/ready;
- detecção de erros.

O pacote utilizado pelo projeto é um protocolo próprio, com campos como tipo, tamanho, source ID, destination ID, sequence number e CRC.

O diferencial é o ambiente de verificação: os autores utilizam **UVM, constrained-random, scoreboard, assertions e functional coverage**. Foram executados mais de 500 testes randomizados e obtida cobertura funcional de 89,3%.

2. Extração das features no FPGA

2.1 — FENIX (<https://arxiv.org/html/2507.14891v3>)

O FENIX é um trabalho de ML em rede, mas **não faz exatamente a extração de features na FPGA**.

A arquitetura é híbrida: utiliza um **switch programável Tofino** para realizar a extração das features e uma **FPGA ZU19EG** para executar a inferência da rede neural.

A ideia é que o switch ASIC faça o processamento de características no próprio data plane. Essas features são enviadas para a FPGA, onde ocorre a inferência DNN.

A FPGA possui um chamado **Model Engine**, responsável pela execução da rede neural, enquanto o sistema também possui um **Data Engine** para controlar o fluxo das features entre o switch e a FPGA.

O trabalho usa um algoritmo probabilístico de token bucket para controlar a taxa de envio das features, justamente porque a capacidade de geração de features do switch e a capacidade de processamento da FPGA são diferentes

é útil para entender a arquitetura geral:

tráfego → feature extraction → ML

2.2 — FPGA-Based Network Traffic Classification Using Machine Learning

(<https://repository.aus.edu/server/api/core/bitstreams/521b0ac8-f711-4cb1-9351-1ea78e7d9d11/content>)

Primeiro os autores estudaram quais features de tráfego eram relevantes para classificação. Eles avaliaram features de nível de pacote e de fluxo e utilizaram métodos de seleção de características. Também estudaram quantos pacotes de um fluxo deveriam ser observados antes da classificação. O resultado foi que aproximadamente **60% dos pacotes do fluxo** apresentavam uma boa relação entre precisão e tempo de processamento.

Depois disso, o trabalho implementa em FPGA um **Random Forest**.

A parte de hardware não recebe diretamente os pacotes Ethernet e calcula todas as features. O foco do acelerador é:

receber as características e realizar a classificação.

A implementação do Random Forest utiliza uma arquitetura altamente **pipeline e paralela**. Cada árvore do Random Forest possui várias comparações, e essas operações são organizadas para que diferentes partes da classificação possam ocorrer simultaneamente.

O hardware foi implementado em uma FPGA e o artigo relata throughput médio de **163,24 Gb/s**, utilizando clock de 35 MHz e considerando o tamanho médio dos pacotes utilizados na avaliação.

Portanto, para o TCC, é importante entender que:

esse artigo é excelente para a etapa posterior à sua Feature Extraction, mas não resolve sozinho o problema de:

pacote → flow → features.

Ele resolve principalmente:

features → ML.

3. Identificação dos fluxos

3.1 — A High-Performance and Accurate FPGA-Based Flow Monitor for 100 Gbps Networks (<https://www.mdpi.com/2079-9292/11/13/1976>)

O objetivo é fazer monitoramento de flows em alta velocidade diretamente na FPGA, sem amostragem. A arquitetura utiliza **Cuckoo Hashing em pipeline** para armazenar e localizar as entradas dos flows. O sistema foi implementado em uma Intel Arria 10 no Intel N3000 e trabalha com uma interface Avalon Stream de 512 bits a 200 MHz, permitindo 100 Gb/s.

O pacote é primeiro analisado para gerar uma **flow key**. Essa chave representa o flow.

Depois existe uma estrutura chamada **flow cache**, que armazena as entradas.

Cada entrada possui:

- flow key;
- flow value;
- flag de validade.

O flow key identifica o fluxo, enquanto o flow value contém as estatísticas associadas ao fluxo.

Para localizar a entrada, os autores utilizam **Cuckoo Hashing**. A vantagem é que uma entrada pode possuir mais de uma posição possível na memória. Isso reduz a probabilidade de uma colisão impedir a inserção.

Um problema do Cuckoo Hashing é que, durante uma inserção, uma entrada pode precisar expulsar outra. Para evitar que isso gere uma quantidade imprevisível de ciclos de hardware, os autores criaram uma **victim memory utilizando distributed RAM**.

As flow entries principais são armazenadas em **Block RAM** da FPGA.

O sistema é dividido em três funções principais:

1. agregação dos pacotes em flows;
2. gerenciamento das flow entries;
3. atualização das flow entries com as estatísticas do pacote.

3.2 — A Modular System for FPGA-Based TCP Flow Processing in High-Speed Networks (https://www.researchgate.net/publication/220759341_A_Modular_System_for_FPGA-Based_TCP_Flow_Processing_in_High-Speed_Networks)

Esse artigo é antigo, mas é muito importante conceitualmente porque trabalha diretamente com **processamento stateful de fluxos TCP em FPGA**.

O sistema desenvolvido é chamado **TCP-Processor**. Ele foi criado para ambientes de alta velocidade em que milhões de flows podem estar ativos simultaneamente. O hardware precisa identificar o flow, recuperar o contexto correspondente, processar o pacote e atualizar novamente o estado.

Além do acompanhamento do estado dos flows, o sistema implementa:

- TCP stream reassembly;
- armazenamento de contexto;
- manipulação de flows;
- interface para aplicações que precisam processar os dados TCP.

A implementação foi feita em FPGA e testada com tráfego real. A versão apresentada operava a 85 MHz e conseguia monitorar cerca de **8 milhões de flows TCP bidirecionais** em OC-48, equivalente a 2,5 Gb/s.

Um detalhe importante é que o sistema utiliza **threading** para esconder a latência da memória. Ou seja, quando uma operação precisa esperar uma informação da memória, o hardware pode trabalhar em outra operação enquanto isso.

referência para entender o conceito de:

flow = pacote + contexto persistente.

4. Armazenamento dos dados

4.1 — Flow Monitoring in High-Speed Networks with 2D Hash Tables (https://users.ece.northwestern.edu/~dtn683/downloads/FPL_2004/fpl.final.flowmonitor.pdf)

Esse trabalho apresenta uma abordagem baseada em **tabelas hash bidimensionais** para monitorar flows em alta velocidade.

A ideia principal é utilizar várias estruturas hash em paralelo para registrar informações dos flows.

O trabalho foi implementado em uma **Xilinx Virtex-II XC2V8000** e utilizou paralelismo da FPGA para conseguir processar tráfego em alta velocidade. A implementação chegou a cerca de 73 Gb/s em determinados cenários.

A principal diferença para o artigo de 2022 do Flow Monitor é que esse trabalho é baseado em **estimativas**. Ele utiliza diferentes técnicas de combinação dos resultados das tabelas hash para estimar as estatísticas dos flows.

Os autores apresentam quatro técnicas de Flow Monitoring Unit:

- Min FMU;
- Median FMU;
- Collision Estimate FMU;
- Hybrid FMU.

A abordagem busca obter informação sobre os flows utilizando estruturas pequenas e paralelas, em vez de manter uma entrada completa para cada flow. ([Ece Iowa State](#))

é interessante para entender o trade-off:

pouca memória + alta velocidade × precisão das informações.

4.2 — A High Throughput Parallel Hash Table on FPGA using XOR-based Memory (<https://arxiv.org/pdf/2108.03390>)

Esse artigo não é específico para redes, mas resolve um problema que vocês também terão: **como implementar uma hash table com várias operações simultâneas na FPGA.**

Os autores implementam uma tabela que suporta:

- busca;
- inserção;
- remoção;
- atualização.

O diferencial é permitir várias consultas por ciclo utilizando múltiplos Processing Elements. Para isso, eles criam uma memória com múltiplas portas lógicas usando **XOR-based multi-ported block memories**.

A arquitetura pode ser escalada para até 16 Processing Elements e alcança throughput de até 5926 MOPS.

a ideia interessante é que uma flow table não precisa necessariamente ser uma RAM simples:

entrada → leitura → processamento → escrita → próxima entrada.

É possível criar estruturas paralelas para aumentar o número de flows que podem ser consultados/atualizados por ciclo.

5. Finalização do fluxo

5.1 — Adaptive Flow Timeout Management in Software-Defined Optical Networks

(<https://www.mdpi.com/2304-6732/11/7/595>)

Esse trabalho é útil para entender a **lógica de timeout**, mas não é uma implementação FPGA.

Os autores trabalham com dois conceitos:

- **idle timeout**: remove o flow depois de determinado período sem receber pacotes;
- **hard timeout**: remove o flow depois de determinado tempo, independentemente da atividade.

O mecanismo analisa o **tempo entre a chegada de pacotes consecutivos** para determinar um valor adequado de timeout.

Além disso, considera a ocupação atual da flow table.

A ideia é que, quando a tabela está pouco ocupada, os flows possam permanecer por mais tempo. Quando a tabela está muito cheia, o sistema fica mais agressivo na remoção dos flows. O trabalho divide a utilização da memória em três regiões e modifica o comportamento do timeout de acordo com essa ocupação.

Para o TCC, isso é interessante porque vocês podem utilizar a mesma ideia conceitual:

último timestamp do flow + tempo de inatividade → decidir se o flow expirou.

5.2 — Demystifying FPGA Hard NoC Performance

(https://www.researchgate.net/publication/389895023_Demystifying_FPGA_Hard_NoC_Performance)

Esse trabalho não tem relação direta com finalização de flows.

Ele estuda o **Network-on-Chip interno de FPGAs modernos**, principalmente Versal, comparando NoCs implementados em lógica programável com NoCs hardened.

O objetivo é entender:

- largura de banda;
- latência;
- topologia;
- posicionamento;
- comunicação entre SLRs;
- acesso à memória.

Os autores mostram, por exemplo, que o NoC vertical pode apresentar latência maior que o horizontal e que NoCs hardened podem reduzir significativamente o uso da lógica programável.